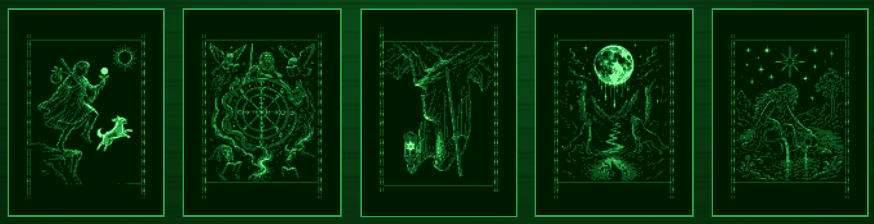
MANUAL DE USUARIO · ESPAÑOL

THE NECRO-ORACLE

"Baraja el mazo. Nombra el repositorio. Los muertos hablarán."



necro-oracle.vercel.app · github.com/FernandoLizana/necro-oracle · commit 5930ae6

NECRO>

En vivo: https://necro-oracle.vercel.app · **Código:** https://github.com/FernandoLizana/necro-oracle · **Video demo:** https://youtu.be/LqITM5tS1Oo

Este manual describe el commit `5930ae6` (main). La sintaxis de los comandos sale del propio parser de la terminal (`packages/ui-crt/src/command/parse.ts`) y de su pantalla de ayuda (`packages/ui-crt/src/render/help.ts`). Los comandos y los textos del oráculo están en inglés, como en la aplicación.

1. Primeros pasos

1. Abre **https://necro-oracle.vercel.app** en un navegador de escritorio o móvil. No necesitas cuenta ni clave.
2. Se reproduce una breve **secuencia de arranque** ("NECROMANTIC SYSTEMS 1986"). Cualquier tecla la salta.
3. Aparece el **menú de inicio**, con el estilo de un cartucho de Game Boy: - **Barra HUD** arriba: `NECRO-ORACLE`, tres corazones y `PY2 CURSE: ACTIVE`. - **LEGACY REPOS**: tres repositorios de Python 2 listos para leer, más `> ENTER ANY REPO...`. - **PREVIEW**: el retrato de la Vidente, tres reversos de carta y un comentario del oráculo sobre el repo seleccionado. - **Barra de ayuda**: `▲▼ SELECT A:SUMMON B:DEMO START:HELP`.
4. Pulsa **Enter** sobre el primer repo. El oráculo lo **invoca** y tira una lectura **Fates** de tres cartas.
5. Lee las cartas, escribe `explain 2` (o `explain present`) para ver la evidencia y luego `resurrect` para revivir el código muerto. Tras una resurrección exitosa, pulsa **LOVERS** (o escribe `lovers <archivo> js`) para unir el Python resucitado a un cliente JavaScript.

El prompt `NECRO>` de abajo recibe los comandos escritos. La **fila de botones** que tiene debajo ofrece cada comando en versión clicable. La línea `SOUL SOURCE` indica quién paga las llamadas de IA: **ORACLE** (la clave del servidor) o **YOURS** (tu propia clave).



El menú de inicio (▲▼ seleccionar · A invocar · B demo · START ayuda)

Controles del menú de inicio

Tecla	Acción
↑ / ↓	Mover la selección (se detiene arriba y abajo, sin dar la vuelta)
Enter o **A**	Activar: invoca el repo seleccionado y lee `fates`. Sobre `> ENTER ANY REPO...` enfoca el prompt
B o **D**	Iniciar la demo sin conexión
H	Mostrar la ayuda

El menú nunca captura teclas pulsadas con Ctrl, Alt o Meta, así que los atajos del navegador siguen funcionando. Solo responde cuando tiene el foco; lo que escribas en `NECRO>` va al prompt. Escribe `home` en cualquier momento para volver al menú.

Los tres repos legacy son `tomaes/pytarl` (DUNGEON · 4 FILES), `ArmstrongJ/robotadventure` (NOIR MYSTERY · 21 FILES) y `neubig/test-repo` (PY2 SPECIMEN · 240 FILES).

2. Comandos

Los comandos no distinguen mayúsculas de minúsculas y los argumentos se separan con espacios. Cada entrada admite hasta 200 caracteres. Lo que la terminal no reconoce responde con `Unknown command: … Type help for the grimoire of commands.`



help: el grimorio de comandos

`summon <owner/repo[#ref|@sha]>`

Fija un repositorio público de GitHub en un commit exacto. Formas aceptadas:

- `summon psf/requests`: usa la rama por defecto
- `summon owner/repo#rama-o-tag`
- `summon owner/repo@<sha-del-commit>`
- `summon https://github.com/owner/repo`: URL completa, con o sin esquema

Atajo: si escribes solo una referencia de repo en la línea (por ejemplo `tomaes/pytarl`), también se invoca. El oráculo responde con `owner/repo @ sha`, el número de archivos y estrellas, y si la instantánea vino de la caché. Los repositorios privados no se pueden leer.

`read [fates|necropsy|celtic]`

Tira una lectura del repo invocado. `read` a secas usa **necropsy**.

Tirada	Cartas	Posiciones
`fates`	3	Past · Present (la aflicción más grave) · Future
`necropsy`	5	The Corpse · The Wound · The Shadow · The Hope · The Call (si hay un ritual de resurrección disponible y por qué)
`celtic`	10	Present · Crossing · Foundation · Recent Past · Crown · Near Future · Self · Environment · Hopes and Fears · Outcome

Cada carta muestra su número, nombre, **orientación** (UPRIGHT = problema activo, REVERSED = riesgo latente o parcial), severidad, una línea mística, el diagnóstico técnico y las evidencias principales. La lectura termina con un indicador **VITALITY 0–100**. Llega por fases: SUMMON, DOWNLOAD, EXTERNAL (registro y OSV) y READING.

Las 12 cartas de diagnóstico: XX Judgement (runtime Python 2), XVI The Tower (dependencias), XV The Devil (seguridad), XIII Death (código muerto), X Wheel of Fortune (tests), IX The Hermit (documentación), XII The Hanged Man (abandono), XIV Temperance (CI y linting), XI Justice (licencia y gobernanza), XVIII The Moon (complejidad), XVII The Star (fortalezas, siempre aparece) y XXI The World (salud general).

`explain <posición|carta>`

Muestra la evidencia completa de una carta. Ejemplos:

- `explain 2`: la carta en la posición 2 de la tirada actual
- `explain past`, `explain present`, `explain future`: alias de `explain 1`, `explain 2` y `explain 3`
- `explain judgement`, `explain hanged_man`: por id de carta

Ids válidos: `judgement tower devil death wheel hermit hanged_man temperance justice moon star world`.

`resurrect [<carta-o-archivo>]`

Revive el código muerto que señalaron las cartas, de Python 2 a Python 3. El botón **RESURRECT** aparece cuando la lectura sacó JUDGEMENT.

- `resurrect`: revive todos los candidatos Python 2 (máximo 50 archivos)
- `resurrect judgement`: por id de carta
- `resurrect src/legacy/tool.py`: un solo archivo (todo argumento que contenga `/` o termine en `.py` se trata como ruta)

Qué ocurre:

1. **Pre_Pass**: seis correctores deterministas reescriben `print` como sentencia, `except E, e:`, `<>`, literales long (`10L`), los métodos de iteración de dict y el uso de unicode/str.
2. Si quedan diagnósticos, la **pasada LLM** envía solo esos fragmentos residuales a un modelo. Usa el nivel FAST y escala a STRONG en los fragmentos difíciles o grandes.
3. Una **compuerta de validación** solo acepta salida que se parsee como Python 3 sin errores y sin construcciones de Python 2. Si no, el archivo se informa como no convertido; el oráculo nunca finge.
4. Se reproduce el **ritual del Necronomicón** (con movimiento completo), y después el **diff**, la **vitalidad antes → después** y un **parche descargable**. En algunas pantallas el panel puede no bajar solo hasta el final: desplaza el panel de salida para llegar al diff y a **DOWNLOAD PATCH**.

`lovers <archivo> js`

Une un archivo Python 3 resucitado con un **cliente JavaScript** (el ritual "Frankenstein"). Ejemplo: `lovers src/lich.py js` (también acepta `javascript`). Judgement enlaza el grimorio `lovers-js`, así que tras un `resurrect` exitoso aparece el chip **LOVERS** en la fila de botones y rellena `lovers <archivo>`. Fases: PLAN → GENERATE → VALIDATE → BIND. El resultado es una vista dividida y un **ZIP** con el "corazón" Python (un despachador JSON-lines), el "amado" JS (un cliente que lanza el corazón), un esquema de contrato JSON y tests de contrato para ambos lados. Si el panel no baja solo hasta el final, desplázalo para llegar a la unión y a **DOWNLOAD PAIR**. Requisitos: la lectura debe marcar el archivo como enlazable (Judgement con `lovers-js`), y el archivo debe tener hasta 8 funciones de nivel superior enlazables y ocupar como máximo 32 KB. Si no, el oráculo lo rechaza con un error tipado sin llamar a ningún modelo. El modo demo ejecuta el mismo ritual con el Mock LLM y sin red.

`tarot [pregunta]`

Saca una **lectura del destino de tres cartas** (Past · Present · Future) de los 22 Arcanos Mayores completos. Las cartas son siempre distintas y cada una sale derecha o invertida de forma independiente, con un generador aleatorio criptográfico. La pregunta es texto libre opcional (hasta 200 caracteres) y se trata estrictamente como dato. Ejemplos: `tarot`, `tarot will my legacy code live?` La revelación es ceremonial: barajar y cortar, cada carta repartida y volteada con una línea de la Vidente, la **lectura determinista sin conexión** escrita párrafo a párrafo y al final un **VERDICT**. Con el movimiento desactivado aparece de inmediato. No hace falta invocar un repo antes. El servicio de lectura con modelo FAST (una sola llamada, ≤400 tokens de salida, tope diario propio) ya está construido y probado, pero **aún no está cableado a una ruta `/api/tarot` en vivo**: figura en el Roadmap; este comando hoy no gasta crédito de IA.

`key set | key status | key clear` (tu propia clave)

Ver la sección 3.

`model <fast|strong> <id>`

Elige qué modelo de OpenRouter usar en cada nivel **con tu propia clave**, solo durante esta sesión. Ejemplo: `model strong anthropic/claude-sonnet-4.5`. El id debe tener la forma `vendor/model` (con `~` inicial y `:tag` opcionales, hasta 128 caracteres y sin espacios). Si no hay clave vinculada, la elección se registra pero queda inactiva: el oráculo sigue con sus propios modelos.

`demo [nombre]`

Reproduce una lectura grabada **sin red**. Nombres: `lich-py2` (un repo Python 2 muerto), `mixed-ruins` (a medio arruinar) y `green-garden` (sano). `demo` a secas muestra el catálogo. La demo se **reproduce sola**: muestra el resumen, tira la lectura `fates` y termina con una tirada de tarot con semilla fija. Cada bloque lleva la etiqueta `DEMO REPLAY · a recorded reading, no live summon.` Escribe `home` para salir, o haz `summon` de un repo real para reemplazarla.

`motion [auto|on|off]`

Controla las animaciones de revelado y del CRT. `auto` sigue la preferencia de "reducir movimiento" de tu sistema. `motion` a secas indica el modo actual ("Motion is …"). La preferencia se recuerda en este navegador.

`sound [on|off]`

Activa o desactiva el zumbido espectral, los efectos de sonido y la música de fondo. El sonido está **desactivado por defecto** y solo empieza después de que interactúes con la página. `sound` a secas indica el estado y `sound on` confirma con "Sound set to on." La preferencia se recuerda en este navegador.

`help`, `clear`, `home`

- `help`: lista todos los comandos y atajos de teclado.
- `clear` (o **Ctrl+L**): limpia la pantalla pero **conserva la lectura** en memoria, así que `explain` y `resurrect` siguen funcionando.
- `home`: vuelve al menú de inicio y sale de cualquier demo.

Fila de botones

`HELP · SUMMON · READ · DEMO · TAROT · CLEAR · MOTION · SOUND · KEY · MODEL`, más botones de contexto: **RESURRECT** aparece cuando la lectura sacó JUDGEMENT, y **LOVERS** cuando Judgement enlazó `lovers-js` y hay un archivo resucitado enlazable listo. Los botones que necesitan argumento (SUMMON, MODEL, LOVERS) rellenan el prompt (por ejemplo `summon`) y lo enfocan para que termines de escribir. El resto se ejecuta de inmediato.

3. BYOK: trae tu propia clave de OpenRouter

Por defecto, el oráculo alojado paga las llamadas de IA (**SOUL SOURCE: ORACLE**) dentro de sus límites diarios. Para usar tu propia cuenta de OpenRouter:

1. Escribe `key set` (o pulsa **KEY**). El oráculo responde *"Whisper your key to the void..."* y se abre un campo **enmascarado**.
2. Pega tu clave de OpenRouter en ese campo y pulsa **Enter**. **Escape** cancela: el campo se borra y el foco vuelve al prompt.
3. El oráculo confirma: *"The key is bound. SOUL SOURCE: YOURS; the last four runes are ..."*.
4. Opcional: elige modelos con `model fast <id>` y `model strong <id>`.
5. `key status` indica si hay una clave vinculada y solo muestra sus **últimos 4 caracteres**.
6. `key clear` olvida la clave al instante y cancela cualquier petición en curso. SOUL SOURCE vuelve a ORACLE.

Cómo se protege tu clave

- **Nunca escribas la clave en la línea de comandos.** Si escribes `key set sk-…`, el oráculo lo intercepta *antes* de que se muestre en pantalla, se guarde en el historial o se trate como comando desconocido. En su lugar abre el campo enmascarado y te advierte: *"Never speak your key aloud on the command line."*
- La clave vive **solo en la memoria de esta pestaña**. No se escribe en localStorage, sessionStorage, cookies ni IndexedDB. Si recargas la página o abres otra pestaña, no hay clave.
- Viaja al servidor en una única cabecera dedicada, se usa **solo para esa petición** y luego se descarta. Nunca se registra en logs, ni se guarda en caché, ni se devuelve en una respuesta, ni forma parte de una clave de caché.
- El endpoint de destino es fijo, así que la clave solo puede ir a OpenRouter.
- Si tu clave es rechazada (inválida o sin crédito), recibes un error claro y tipado. El oráculo **nunca recurre en silencio** a la clave del servidor.
- Con tu propia clave no te afectan los topes diarios de gasto del servidor; solo pagas tus propias llamadas.

4. Límites del modo servidor (SOUL SOURCE: ORACLE)

Para que el oráculo gratuito siga disponible para todos:

Límite	Valor
Invocaciones por IP y hora	20
Lecturas por IP y hora	10
Resurrecciones por IP y hora	5
Uniones Lovers por IP y hora	5
Tope diario de peticiones (global)	1000 por defecto
Topes diarios de IA del servidor / Lovers / tarot	1000 / 200 / 300 por defecto, cada uno con su propio contador
Llamadas LLM por lectura	12 por defecto (máximo 2 por unidad: FAST y luego un reintento escalado)
Tamaño del repositorio	50 MB comprimido / 200 MB descomprimido
Archivos por ritual	hasta 50 candidatos

Al llegar a un límite verás `ERROR RATE_LIMITED` ("The gate is barred for a little while.") con un tiempo de espera. Cuando se agota el presupuesto diario de IA del servidor, el oráculo **degrada con suavidad**: los resultados deterministas se siguen mostrando y el tarot usa su texto sin conexión. Si vinculas tu propia clave, los topes de IA del servidor dejan de aplicarse a ti.

5. Sonido y música

Todo el audio se **sintetiza en tiempo real** con la Web Audio API; no hay archivos de audio.

- `sound on` activa un zumbido espectral grave, efectos breves (invocación y revelado) y un chiptune original en bucle en La menor (Am–F–C–G, con voces de onda cuadrada y triangular).
- El volumen tiene un tope por diseño. El sonido nunca suena antes de que interactúes, y el zumbido ambiental se calla en modo de movimiento reducido.
- `sound off` silencia todo al instante.

6. Modo demo y parámetros de URL

Parámetro	Efecto
`?boot=skip`	Salta la secuencia de arranque y va directo al menú
`?ritual=force`	Reproduce siempre el ritual completo del Necronomicón, incluso con `?boot=skip` o en ejecuciones automatizadas

Ejemplo: `https://necro-oracle.vercel.app/?boot=skip&ritual=force`

La demo (`demo lich-py2`, o **B** en el menú) funciona sin conexión y es determinista: las mismas cartas y los mismos glitches cada vez. Es útil para presentaciones o cuando la red no es fiable.

7. Atajos de teclado

Tecla	Acción
Enter	Ejecutar el comando escrito
↑ / ↓ (en el prompt)	Recorrer el historial de comandos
Ctrl+L	Limpiar la pantalla (la lectura se conserva)
Escape	Saltar el revelado o la animación en curso; cancelar el campo de clave
Ctrl+C	Abortar la invocación o la lectura en curso
↑ ↓ · Enter/A · B/D · H (menú)	Navegar, invocar, demo, ayuda
Tab	Orden de foco normal del navegador (el foco nunca queda atrapado)

8. Errores y solución de problemas

Los errores se muestran en dos capas: una línea mística breve y luego el dato técnico en lenguaje claro.

Código	Significado	Qué hacer
`INVALID_REPO_URL`	Referencia de repo mal formada	Usa `owner/repo` o una URL `github.com/owner/repo`
`REPO_NOT_FOUND`	No existe ese repo público	Revisa la ortografía
`REPO_PRIVATE`	El repo es privado	Solo se pueden leer repos públicos
`REF_NOT_FOUND`	La rama, el tag o el sha no existen	Omite la ref para leer la rama por defecto
`DOWNLOAD_TOO_LARGE` / `FILE_TOO_LARGE`	El repo o un archivo supera los límites de tamaño	Prueba un repo más pequeño o resucita un solo archivo
`RATE_LIMITED`	Se alcanzó un límite por IP o diario	Espera el tiempo indicado o vincula tu propia clave
`NETWORK_TIMEOUT`	La conexión falló o expiró	Revisa tu conexión y reintenta, o usa `demo`
`UPSTREAM_UNAVAILABLE`	GitHub o el proveedor de IA no responden, o tu clave no tiene crédito	Reintenta más tarde o recarga crédito en OpenRouter
`AUTH_INVALID`	Tu clave de OpenRouter fue rechazada	`key clear` y luego `key set` con una clave válida
`VALIDATION_FAILED`	Una respuesta no pasó la validación de esquema, o no se cumplió un requisito previo (p. ej. `lovers` sobre un archivo no enlazable)	Repite la lectura y reintenta
`PARSER_INIT_FAILED`	El motor de análisis de código no pudo arrancar en el servidor	Raro tras el arreglo de gramáticas en `619ebc4`. Reintenta; si persiste, usa `demo lich-py2` y repórtalo

Código	Significado	Qué hacer
`CARD_NOT_FOUND`	Id de carta desconocido en `explain`	Usa uno de los 12 ids de la lista

Otros consejos:

- **"Unknown command"**: revisa la sintaxis con `help`. Cada comando espera un número exacto de argumentos (p. ej. `lovers <archivo> js` necesita los dos).
- **Nada que explicar o resucitar**: primero invoca un repo y haz `read`.
- **Un ritual a la vez**: espera a que termine el actual o pulsa Ctrl+C.
- **No hay sonido**: escribe `sound on`, interactúa una vez con la página y comprueba que la pestaña no esté silenciada.
- **Demasiado movimiento**: `motion off`, o activa "reducir movimiento" en tu sistema y usa `motion auto`.
- **No ves el diff o la descarga después de resurrect / Lovers**: desplaza el panel de salida. En algunas pantallas la vista de resultado aún no baja sola hasta el final; el diff y **DOWNLOAD PATCH** / **DOWNLOAD PAIR** están justo debajo.

---

9. Preguntas frecuentes

¿El oráculo ejecuta el código de mi repositorio? No. El contenido solo se analiza (tree-sitter y parsers de manifiestos) y se trata como dato no confiable. Nunca se ejecuta, se importa ni se evalúa.

¿Puede la IA inventarse un diagnóstico? No. Cada carta la saca un detector determinista con evidencia. La IA solo reescribe el código residual, y toda su salida debe pasar compuertas de validación.

¿Se guarda mi clave? No. Vive solo en la memoria de esta pestaña y se usa petición a petición. Ver la sección 3.

¿Abre pull requests en mi repo? No. La resurrección produce un diff y un parche descargable que aplicas tú.

¿Qué lenguajes soporta? El diagnóstico y la resurrección son para Python (de 2 a 3). La unión Lovers genera un cliente JavaScript para código Python 3.

¿Funciona sin conexión? Las demos (lectura, resurrect y Lovers con el Mock LLM) y la lectura de tarot determinista, sí. Las invocaciones, resurrecciones y uniones Lovers en vivo necesitan red (y una clave de OpenRouter para las pasadas LLM).

¿Es accesible? Sí. Usa texto real en el DOM con una única región viva para lectores de pantalla, se maneja por completo con teclado, respeta el movimiento reducido, no tiene violaciones de axe y funciona a 320 px de ancho y con zoom del 200 %.

¿De dónde sale el arte de las cartas? Es un tratamiento pixel art del mazo Rider–Waite–Smith (Pamela Colman Smith, 1909, dominio público).

---

10. Desarrollo local (para jueces que quieran ejecutarlo)

Requisitos: **Node 22** y **pnpm 10** (el repo fija `pnpm@10.11.0` en `packageManager`).

```
git clone https://github.com/FernandoLizana/necro-oracle
cd necro-oracle
pnpm install
pnpm -r build            # compila todos los paquetes (la web importa su salida dist)
pnpm dev                 # servidor de desarrollo Next.js de apps/web (http://localhost:3000)
```

No hace falta ninguna variable de entorno. Sin claves, la app usa el proveedor determinista **Mock**: las lecturas se muestran, la resurrección ejecuta el Pre_Pass determinista y el tarot usa su lectura sin conexión. Variables opcionales del servidor (ver `docs/MODELS.md`): `OPENROUTER_API_KEY`, `NECRO_LLM_MODEL_FAST`, `NECRO_LLM_MODEL_STRONG`, `NECRO_LLM_MODEL_FALLBACKS`, `GITHUB_TOKEN`, `UPSTASH_REDIS_REST_URL` / `_TOKEN`, `NECRO_PUBLIC_URL` y los límites `NECRO_*_CEILING`. Nunca subas un archivo `.env*`; un hook de Kiro bloquea que el agente los escriba.

Controles de calidad:

```
pnpm test                                # Vitest: tests unitarios + de propiedades
pnpm typecheck                           # tsc (referencias de proyecto) + web
pnpm lint                                # ESLint, --max-warnings=0
pnpm -r build                            # todos los paquetes + web
node scripts/check-web-budget.mjs        # presupuesto 250 KB gz, chunks separados, marcadores
de fuga
pnpm --filter @necro/web check:wasm-trace  # WASM de tree-sitter empaquetado en las funciones
pnpm --filter @necro/web exec playwright install   # una vez
pnpm e2e                                 # Playwright e2e + axe
pnpm smoke:llm                           # tuberías LLM, sin red por defecto
pnpm demo:bundle                         # regenera el paquete de la demo sin conexión
```

Estado en `5930ae6`: 3.091 tests unitarios y de propiedades en verde (301 archivos), 52 tests e2e de Playwright en verde, typecheck limpio, lint con 0 advertencias, y build, presupuesto y comprobación WASM consciente de symlinks en verde. Producción en https://necro-oracle.vercel.app corre Node 22.x; lectura, resurrect y Lovers en vivo se verificaron el 2026-11-09 contra `tomaes/pytarl`: `resurrect pytarl.py` subió la vitalidad **0 → 12** sin coste de LLM (Pre_Pass determinista); `lovers pytarl.py javascript` unió **7 funciones** por **6.418 tokens** (**0,0842 USD**), tras una corrida anterior de Lovers en el mismo archivo con `anthropic/claude-sonnet-4.5` por **0,019 USD**.